

CuPINN: Optimizing PINNs through curvature minimization and residual landscape flattening

Raouf Zare Moayed, Mostafa Abbaszadeh^{*}, Mehdi Dehghan^{ID}

Department of Applied Mathematics, Faculty of Mathematics and Computer Sciences, Amirkabir University of Technology (Tehran Polytechnic), No. 424, Hafez Ave., 15914 Tehran, Iran

ARTICLE INFO

Keywords:

Physics-informed neural networks
Partial differential equations
Residual minimization
Curvature minimization

ABSTRACT

Physics-Informed Neural Networks (PINNs) have gained recognition as a powerful framework for solving partial differential equations (PDEs) by incorporating the PDE residual directly into the neural network's loss function. However, while effective in many cases, standard PINNs often struggle with accuracy. *Gradient-Enhanced PINNs (GPINNs)* were developed to address these issues by enforcing zero residual gradients, but they have not been able to sufficiently guide the learning process in many demanding physical applications. In this work, we introduce two novel methods, RCuPINN, and CuPINN, which significantly improves both the accuracy and training efficiency of PINNs. These methods are built on the key observation that the residual over the entire domain can be viewed as a manifold in $\mathbb{R}^{d+1}$, where d is the dimensionality of the problem. Ideally, this manifold should exhibit zero value across the domain with minimal curvature in regions where the solution is smooth. By enforcing a smooth and flat residual manifold with zero curvature, we introduce a stronger inductive bias that promotes more accurate residual learning, backed by theoretical analysis. Furthermore, we demonstrate that minimizing the curvature inherently reduces the gradient, which further enhances the solution's accuracy. To achieve this, we leverage Hutchinson Trace Estimation to compute the eigenvalues of the curvature, allowing us to directly minimize the curvature of the residual surface. This refined loss function leads to more efficient training and improved accuracy. Additionally, we combine our method with GPINNs to create hybrid approaches, RCuGPINN, CuGPINN. Extensive experiments on both linear and nonlinear PDEs demonstrate that our methods achieve residual and approximation errors on the order of 10^{-1} compared to existing approaches, including GPINNs. This substantial reduction in error illustrates the superior accuracy and effectiveness of our methods.

1. Introduction

Partial differential equations (PDEs) are fundamental tools for modeling a wide range of phenomena in science and engineering, such as fluid dynamics, heat transfer, financial modeling, and materials science. The ability to solve PDEs accurately and efficiently is crucial for advancing both theoretical understanding and practical applications in these fields. However, traditional numerical methods often face several challenges, including high computational costs, the need for complex meshing, numerical instabilities, and limited flexibility in handling diverse problem domains.

Deep neural networks, with their remarkable approximation capabilities [1], have revolutionized many fields. This success has sparked growing interest in scientific machine learning, which integrates physical structures and domain knowledge into the learning

^{*} Corresponding author.

E-mail addresses: raoufmoayed2000@gmail.com (R.Z. Moayed), m.abbaszadeh@aut.ac.ir (M. Abbaszadeh), mdehghan@aut.ac.ir (M. Dehghan).

<https://doi.org/10.1016/j.cma.2025.118180>

Received 25 January 2025; Received in revised form 22 June 2025; Accepted 22 June 2025

Available online 14 July 2025

0045-7825/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

process. By embedding physical laws directly into the training process, DNNs enhance our ability to model and predict complex phenomena. The idea of Physics-Informed Neural Networks (PINNs) [2]—has emerged as a promising alternative to traditional numerical PDE solvers. PINNs embed PDEs directly into the neural network's loss function through automatic differentiation, enabling the calculation of gradients and other necessary terms for solving PDEs. This approach overcomes many of the limitations associated with conventional methods. Their mesh-free nature allows them to solve a wide range of PDEs in complex domains. Additionally, PINNs can easily handle different types of constraints, incorporate data, and solve both linear and non-linear problems due to the universal approximation theorem [1] and the flexibility of activation functions.

Despite these advancements, several challenges persist in improving PINNs' accuracy and efficiency. The core of PINN methodology lies in the residual, which reflects how well the network has learned the underlying physical laws. Ideally, the residual should be zero at all points in the domain. However, early in training, one trivial solution that achieves zero residual is a network with all weights set to zero. At the same time, the boundary and initial condition losses push the model in a different direction. Therefore, accurately balancing the solution error and the residual is critical for capturing the underlying physics of the system.

Recent work [3] has emphasized optimizing both the residual and its gradient, arguing that minimizing the residual alone may not be sufficient. For instance, zero gradients can drive the model prematurely toward zero, introducing a harsh bias. Moreover, making the residual gradient zero does not account for behavior in the local neighborhood of each point. To address these challenges, we propose a new family of algorithms – CuPINN, RCuPINN, and their gradient-augmented variants CuGPINN and RCuGPINN – that go beyond gradient minimization and instead target curvature minimization. By promoting a smoother residual landscape, these methods improve stability and accuracy, especially in capturing complex physical phenomena. Our core idea is to enforce zero curvature on the residual surface, which regularizes the local behavior of the residual and avoids trivial solutions. We achieve this efficiently by estimating the curvature using the Hutchinson trace estimator and finite difference approximations, enabling scalable training without computing the full Hessian. Furthermore, we balance the impact of the gradient term to avoid collapse toward the zero solution. Our experiments demonstrate the superiority of these methods across various PDE benchmarks, including high-dimensional problems. We show consistent improvements in accuracy, residual smoothness, and training stability over existing methods like GPINNs.

Our main contributions are summarized as follows:

- We introduce four innovative methods, **RCuPINN**, **CuPINN**, **RCuGPINN** and **CuGPINN**, designed to flatten the residual surface. In an ideal scenario, this surface should approach zero, indicating improved accuracy. Our theoretical analysis supports the effectiveness of these methods, and we provide extensive evidence demonstrating their superior performance compared to traditional PINN and GPINN approaches across seven diverse problems.
- We show that our method significantly outperforms others in low-data regimes, particularly in cases where both PINN and GPINN fail to provide accurate solutions.
- Through experimental results, we demonstrate that our methods consistently produce a much flatter residual surface than other approaches.
- We show that minimizing the sum of the eigenvalues of the Hessian also implicitly minimizes the gradient, leading to further improvements in performance.

The structure of the paper is as follows: In Section 2, we review the related work. Section 3 presents the preliminaries, followed by a detailed description of our proposed method. The theoretical justification and resulting implications of our approach are discussed in Section 4. We present experimental results in Section 5, and provide additional analysis in Section 6. Finally, we conclude the paper in Section 7.

2. Related work

Physics-Informed Machine Learning. In this work, we employ Physics-Informed Neural Networks (PINNs) [2], which solve partial differential equations (PDEs) by minimizing boundary and residual losses. PINNs are grounded in a robust theoretical framework, encompassing generalization bounds and convergence rate analyses [4–8]. These models have been successfully applied to various challenging problems, including fractional PDEs, [9], integro-differential equations [10], etc.

PINNs have demonstrated their versatility across many fields in computational science, such as nonlinear PDEs [11], solid mechanics [12], uncertainty quantification [13], and fluid mechanics [14,15]. However, one significant challenge in PINNs is accurately resolving complex domains and discontinuities, which are inherently difficult to capture. To address this issue, several studies have focused on resampling techniques in regions with high residuals. For instance, the residual-based adaptive refinement (RAR) method [10] iteratively introduces new collocation points in areas with large residuals. Similarly, adaptive sampling methods (ASMs) [16] subdivide the computational domain and refine the sampling process based on the mean residual and solution gradient within each sub-domain. Additionally, [17,18] presented a strategy that iteratively adds new data points by following the gradient information of the loss function with respect to residual datapoints, prioritizing those with the highest residuals to improve model performance.

Addressing high-dimensional PDEs poses another challenge, particularly due to the curse of dimensionality that arises in fields such as mathematical finance (e.g., the Black–Scholes equation [19–21]) and optimal control (e.g., the Hamilton–Jacobi–Bellman equation [22–24]). These high-dimensional problems substantially increase computational complexity and training times. Several studies have explored techniques to mitigate these challenges [20,25]. In [25], a method was introduced for solving second-order

parabolic equations by reducing the computational cost of calculating the Hessian – typically required for backpropagation – using Hutchinson's trace estimator [26]. This method provides an efficient estimate of second-order derivatives, significantly reducing both time and computational resources.

Priors and Constraints in Neural Networks. Another line of research focuses on incorporating inductive biases into the architecture, loss function, or optimization procedure of neural networks to enhance performance. Some works target the optimization process itself. For instance, in [27], researchers framed the problem as a min–max optimization to increase model robustness against perturbations, leading to more stable solutions. This approach models the interaction between adversarial perturbations and network optimization, ultimately improving the robustness of the model.

In addition to solving specific PDEs, alternative approaches like Neural Operator [28,29], DeepONet [30], Fourier neural operators [31] aim to learn the operators governing PDEs directly. A DeepONet consists of two sub-networks: a branch net that encodes the input function at a fixed number of sensor points, and a trunk net that encodes the spatial locations for the output functions. This architecture allows DeepONet to approximate infinite-dimensional operators, effectively expanding the scope of neural networks beyond traditional PDE-solving methods by directly learning the underlying operators of the system. DeepONet has been successfully applied to a wide range of operator-learning tasks, such as fluid dynamics, structural mechanics, and control systems, offering a flexible and scalable alternative to PINNs.

Another class of networks embeds domain-specific information directly into the model architecture, drawing on insights from physical principles. For example, in [32], Hamiltonian Neural Networks (HNNs) were proposed, inspired by Hamiltonian mechanics—a branch of physics that deals with conservation laws and invariances. By embedding the Hamiltonian structure into the network, HNNs ensure that physical properties such as energy conservation are respected throughout the learning process, leading to models that are better aligned with the underlying physical system. Similarly, Lagrangian Neural Networks (LNNs) [33] were developed to work in arbitrary coordinate systems, overcoming the limitations of Hamiltonians that require canonical coordinates. LNNs extend the concept by leveraging the Lagrangian formulation of mechanics, which provides more flexibility in representing systems with complex constraints or nonlinear dynamics.

Several methods modify the loss function to incorporate priors for more effective learning. For instance, in [3], the authors introduce a term for the gradient norm of residual datapoints, enforcing a stronger bias toward achieving near-zero residuals. In [34], the authors propose a causality-respecting approach for temporal problems, where residuals at time t are minimized only after ensuring that residuals at previous times have been adequately learned. Similarly, in [35], the authors suggest assigning a learnable weight to each residual datapoint, allowing the model to prioritize minimizing the loss on the most important datapoints.

These approaches illustrate the growing trend of embedding physical principles and mathematical structures directly into neural networks to improve their interpretability, accuracy, and generalization capabilities across a range of scientific and engineering domains.

3. Proposed method

In this section, we introduce a novel methodology designed to enhance the performance of Physics-Informed Neural Networks (PINNs). Our approach addresses key limitations of traditional PINNs by focusing on optimizing the residual loss surface to ensure more accurate and efficient solutions to partial differential equations (PDEs). Specifically, we aim to enforce a flat residual surface by driving all eigenvalues of the Hessian of the residual loss with respect to inputs toward zero, thus achieving a zero-curvature surface. To incorporate this into training, we minimize the sum of squared eigenvalues of the residual Hessian. Since computing the full Hessian is expensive, we use the Hutchinson trace estimator [26] to efficiently approximate its trace via random vectors, and we estimate Hessian-vector products using finite differences. This curvature minimization is integrated into the overall loss function, enabling the model to regularize the local geometry of the residual. To further improve robustness, we combine this curvature loss with the gradient penalty used in GPINNs and introduce weighted terms to balance expressiveness and optimization stability. In particular, we down-weight the gradient term (e.g., by a factor of 0.1) to prevent the model from collapsing into trivial solutions. We denote the resulting methods as CuPINN and RCuPINN (for curvature-only models), and CuGPINN and RCuGPINN (for curvature + gradient-enhanced models). These algorithms are tested across a range of PDEs in up to four dimensions and consistently outperform baseline methods in terms of residual loss, approximation error, and residual surface smoothness.

3.1. Preliminaries

3.1.1. Physics-informed neural networks (PINNs)

This paper focuses on employing PINNs [2] for solving partial differential equations (PDEs) defined on the domain $\Omega \subset \mathbb{R}^d$ with the boundary/initial condition on Γ and the residual condition within Ω :

$$\mathcal{L}u(x) = g(x), \text{ in } \Omega, \quad Bu(x) = B(x), \text{ on } \Gamma. \quad (3.1)$$

Given the boundary points $\{x_{b,i}\}_{i=1}^{n_b} \subset \Gamma$ and the residual points $\{x_{r,i}\}_{i=1}^{n_r} \subset \Omega$, PINNs minimize the discrepancy in the residual and on the boundary for the output of the model u_θ parameterized by θ , note that $x_{data} = \{x_b\} \cup \{x_r\}$:

$$\begin{aligned} \mathcal{L}(\theta; x_{data}) &= \lambda_b \mathcal{L}_b(\theta; x_b) + \lambda_r \mathcal{L}_r(\theta; x_r) \\ &= \frac{\lambda_b}{n_b} \sum_{i=1}^{n_b} |Bu_\theta(x_{b,i}) - B(x_{b,i})|^2 \\ &\quad + \frac{\lambda_r}{n_r} \sum_{i=1}^{n_r} |\mathcal{L}u_\theta(x_{r,i}) - g(x_{r,i})|^2, \end{aligned}$$

where λ_b and λ_r are the boundary and residual loss weights, respectively.

3.1.2. Hutchinson trace estimation (HTE)

The Hutchinson trace estimator is a probabilistic method used to estimate the trace of large matrices, particularly in the context of large-scale computations where direct calculation is infeasible. The trace of a matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$ can be randomly estimated as follows:

$$\text{Tr}(\mathbf{A}) = \mathbb{E}_{v \sim p(v)} [v^T \mathbf{A} v],$$

For all random variable $v \in \mathbb{R}^d$ such that $\mathbb{E}_{v \sim p(v)} [v v^T] = \mathbf{I}$. This property is crucial for the unbiased estimation of the trace. To compute the trace using the Hutchinson estimator, we employ a Monte Carlo approach:

$$\text{Tr}(\mathbf{A}) \approx \frac{1}{V} \sum_{i=1}^V v_i^T \mathbf{A} v_i,$$

where each $v_i \in \mathbb{R}^d$ are independent and identically distributed (i.i.d.) samples from the distribution $p(v)$. The choice of distribution $p(v)$ is flexible; however, one of the most common choices is the standard normal distribution, which provides good convergence properties and is computationally efficient. Another viable option for the distribution $p(v)$ is the Rademacher distribution, which assigns equal probability to +1 and -1.

3.1.3. Hessian vector product

The Hessian matrix $\mathbf{H}$ of a scalar-valued function $f(\cdot)$ with respect to its parameters $\mathbf{x}$ contains second-order partial derivatives. Computing the full Hessian matrix is often impractical due to its size and computational cost. Instead, the Hessian-vector product $\mathbf{H}v$ can be approximated using finite differences:

$$\mathbf{H}v \approx \frac{\nabla f(x) - \nabla f(x + \epsilon v)}{\epsilon}, \quad (3.2)$$

where $\nabla f(x)$ denotes the gradient of f with respect to x , and ϵ is a small perturbation parameter. This approximation allows us to estimate the product of the Hessian matrix on a vector v without explicitly forming or storing the entire Hessian matrix $\mathbf{H}$. This approximation is particularly useful in large-scale optimization scenarios where memory efficiency and computational speed are critical. By avoiding the need to form or store the entire Hessian matrix, we can achieve significant improvements in both efficiency and scalability.

3.2. Motivation

As explained in the previous section, the residual term in Physics-Informed Neural Networks (PINNs) represents the differential equation, and ideally, this residual should be zero across the entire domain Ω . This means that the residual surface should be flat and smooth, with zero curvature. Achieving not only a zero residual but also zero curvature in the local neighborhood around residual datapoints can greatly improve the neural network's accuracy and stability in modeling complex physical systems. To put this in mathematical terms, the Hessian matrix measures how the gradient of the function changes. If all the eigenvalues of the Hessian are zero, this indicates that the gradient remains constant, implying that the function does not bend or curve, resulting in a flat surface. By driving each eigenvalue of the Hessian toward zero, we explicitly enforce this flatness, ensuring that the residual surface remains smooth and well-behaved. Furthermore, our approach aligns the gradients in the local neighborhood around datapoints, allowing the network to capture complex behaviors without forcing the model toward trivial solutions with zero outputs. This provides a better bias for learning, improving both the model's performance and generalization.

3.3. Residual surface flattening

To minimize the curvature, we adopt the approach outlined in [36]. Let $\lambda_1, \dots, \lambda_n$ denote the eigenvalues of the Hessian of the residual function $L_r(\theta; x_r)$ with respect to the inputs. Our goal is to achieve $\sum_{i=1}^n \lambda_i^2 = 0$, which necessitates that $\lambda_i = 0$ for all i . Furthermore, it is known that:

$$\sum \lambda_i^2 = \text{Tr}(\mathbf{H}_{L_r}^2(x)).$$

So using HTE we have:

$$\text{Tr}(\mathbf{H}_{L_r}^2(x)) = \mathbb{E}_{v \sim P(v)} [v^T \mathbf{H}_{L_r}^2 v].$$

Using the law of large numbers, we approximate:

$$\text{Tr}(\mathbf{H}_{L_r}^2(x)) \approx \frac{1}{n} \sum_{i=1}^n \|\mathbf{H}_{L_r} v_i\|_2^2.$$

Instead of sampling multiple directions, we now consider two cases: using a random direction and using the gradient direction, defined as $v = \frac{\nabla L_r(x)}{\|\nabla L_r(x)\|}$. For the random direction case, we perform all experiments using a single sampled direction. This approach is computationally more efficient than using the gradient direction, as it avoids the need to compute gradients. In practice, we

observe that the random direction yields results comparable to those obtained with the gradient direction—there is no significant performance difference between the two methods. For the gradient direction, as discussed in Section 6, we empirically show that, across all problems, the cosine similarity between the gradient and the eigenvector corresponding to the largest eigenvalue is consistently at least 0.70. We also provide a theoretical intuition supporting this observation: the eigenvector of the largest eigenvalue tends to align with the gradient because the gradient indicates the direction of the steepest change in the function. This makes it a natural and effective choice for our analysis. Furthermore, it has been shown in [37] that the Rademacher distribution minimizes the variance of this estimator. Therefore, in our approach, we adopt the Rademacher distribution and take into account the sign of the direction.

Thus, we only need to calculate:

$$\|\mathbf{H}v\|_2^2.$$

Now using the finite difference estimation we could say

$$\mathbf{H}v \approx \frac{\nabla L_r(x) - \nabla L_r(x + \epsilon v)}{\epsilon}. \quad (3.3)$$

So we just need to minimize:

$$\|\nabla L_r(\theta; x_r) - \nabla L_r(\theta; x_r + \epsilon v)\|_2^2. \quad (3.4)$$

Algorithm 1 CuPINN & CuGPINN Algorithm

Inputs:

Iterations: T , Boundary Loss Weight: λ_b , Residual Loss Weight: λ_r , Curvature Loss Weight: λ_c

Gradient Norm Weight: λ_g , Use Gradient Regularization: **gradient**

Boundary Loss: $\mathcal{L}_b$, Boundary Points: x_b , Residual Loss: $\mathcal{L}_r$, Residual Points: x_r

Finite Step: ϵ , Model Parameters: θ

for $t = 1 \dots T$ **do**

$loss = 0$

// Compute Boundary and Residual Losses

$loss = loss + \lambda_b \mathcal{L}_b(x_b)$

$loss = loss + \lambda_r \mathcal{L}_r(x_r)$

if **gradient** **then**

// Gradient Norm Regularization

$loss = loss + \lambda_g \|\nabla_x \mathcal{L}_r(x_r)\|_2$

end if

// Curvature Approximation

$loss = loss + \lambda_c \|\nabla_x \mathcal{L}_r(x_r) - \nabla_x \mathcal{L}_r(x_r + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}_r))\|_2$

// Update Model Parameters

$\theta = \text{optimizer}(\theta, \nabla_{\theta} loss)$

end for

Algorithm 2 RCuPINN & RCuGPINN Algorithm

Inputs:

Iterations: T , Boundary Loss Weight: λ_b , Residual Loss Weight: λ_r , Gradient Loss Weight: λ_g , Curvature Loss Weight: λ_c

Boundary Loss: $\mathcal{L}_b$, Boundary Points: x_b , Residual Loss: $\mathcal{L}_r$, Residual Points: x_r

Finite Step: ϵ , Model Parameters: θ , Gradient Flag: **gradient**

for $t = 1 \dots T$ **do**

$v \sim \text{Rademacher}(\pm 1)$

$x_{\text{random}} = x_r + \epsilon \cdot v$

$loss = 0$

// Compute Boundary and Residual Losses

$loss = loss + \lambda_b \mathcal{L}_b(x_b)$

$loss = loss + \lambda_r \mathcal{L}_r(x_r)$

if **gradient** **then**

// Gradient Norm Regularization

$loss = loss + \lambda_g \|\nabla_x \mathcal{L}_r(x_r)\|_2$

end if

// Curvature Approximation

$loss = loss + \lambda_c \|\nabla_x \mathcal{L}_r(x_r) - \nabla_x \mathcal{L}_r(x_{\text{random}})\|_2$

// Update Model Parameters

$\theta = \text{optimizer}(\theta, \nabla_{\theta} loss)$

end for

We introduce four novel methods for optimizing residual data points and enhancing the learning of physical laws. The first two methods – CuPINN and CuGPINN – focus on minimizing the curvature in the neighborhood around each residual data point using the gradient direction. When the **gradient** flag is set to True, an additional gradient norm regularization term is added to the loss function. The complete algorithm is provided in 1. The other two methods – RCuPINN and RCuGPINN – minimize the curvature using

Table 1
Experimental configurations for all equations.

Equations		Architecture		Training setting			
		Width	Depth	Optimization	Learning rate	Iterations	Finite step
Damped Harmonic Oscillator Equation	5.2	30	3	Adam	0.001	15 000	0.01
1D Poisson Equation	5.3	20	4	Adam	0.001	20 000	0.05
Diffusion Equation	5.4	30	4	Adam	0.001	15 000	0.02
2D Poisson Equation	5.5	30	3	Adam	0.001	15 000	0.2
Kuramoto-Sivashinsky Equation (T=1,10)	5.6	30	3	Adam	0.001	6000	0.01, 0.1
Schrödinger Equation	5.7	30	4	Adam	0.001	30 000	0.05
Burger's Equation	5.8	30	4	Adam	0.001	20 000	0.05

a random direction. Similarly, if `gradient` is `True`, the gradient norm is also minimized. The corresponding algorithm is detailed in 2. The key distinction between these methods lies in the choice of direction: either random or gradient-based. Additionally, the hyperparameter ϵ plays a crucial role in controlling the curvature within the ϵ -neighborhood of each residual data point. Our methods ensure proper gradient alignment within this neighborhood. Details on the specific values of ϵ used for each equation are provided in Table 1, where it is referred to as the *finite step* throughout the paper.

4. Theoretical justification

In this section, we delve into the theoretical foundations of our approach, establishing the relationship between our algorithms and GPINN and providing justification for why minimizing the curvature around residual points not only leads to a reduction in the gradient but also allows the gradient to approach zero naturally. Unlike GPINN, which enforces zero gradient directly through the loss function, our algorithms achieve this with less restrictive assumptions, resulting in a smoother and more flexible learning process. This section also introduces the role of curvature and explains how controlling it in the local neighborhood around residual points enables more accurate and stable learning, even in high-dimensional spaces with sparse data. By minimizing the curvature at residual points, we ensure that the residual is minimized within their neighborhoods, a guarantee that previous methods do not offer. Finally, we employ Taylor's theorem to provide deeper insights into how our method ensures smoother residuals in unsampled areas, improving generalization in those regions.

Relation between our algorithms, and GPINN. Consider the residual function $r(x) = \mathcal{L}u(x) - g(x)$, where our goal is to minimize this residual in order to capture the underlying physical laws. Typically, the loss function used to reduce the residual is $L(x) = r(x)^2$. However, in this section, without loss of generality, we consider $L(x) = \frac{1}{2}r(x)^2$ for convenience. Now, we aim to establish a connection between GPINN and minimizing curvature. It is known that the gradient of the loss function is:

$$\nabla L(x) = r(x)\nabla r(x),$$

and the Hessian of the loss is given by:

$$\nabla^2 L(x) = \nabla r(x)\nabla r(x)^T + r(x)\nabla^2 r(x).$$

In all of our algorithms, the objective is to flatten the residual loss surface by minimizing the eigenvalues of the Hessian, which corresponds to minimizing $\text{Tr}(\nabla^2 L(x))$. Since the trace operator Tr is linear, we have:

$$\begin{aligned}\text{Tr}(\nabla^2 L(x)) &= \text{Tr}(\nabla r(x)\nabla r(x)^T) + r(x)\text{Tr}(\nabla^2 r(x)) \\ &= \|\nabla r(x)\|^2 + r(x)\text{Tr}(\nabla^2 r(x)).\end{aligned}$$

By minimizing the above expression, we are effectively reducing the components of $\|\nabla L(x)\|$, and as a result, minimizing the gradient of the loss. This shows that our algorithms are sufficient to learn a smoother residual surface, improving the overall learning of the physical system.

Role Of Curvature. In Physics-Informed Neural Networks (PINNs), the objective is to learn physical laws by minimizing the residual function, which ideally should be zero across the entire domain. This would result in a perfectly flat residual surface. However, due to the limited availability of data points and the increasing complexity of problems, particularly in higher dimensions, achieving zero residual across the entire domain becomes increasingly challenging. To address this, we propose a method that not only improves residual learning at the sampled points but also minimizes the residual in the local neighborhood around those points, ensuring more accurate results in unsampled regions. To begin, we use Taylor's theorem to analyze the residual function. For a fixed data point x , we have [38]:

$$f(x+v) = f(x) + \nabla f(x)^T v + \frac{1}{2}v^T \nabla^2 f(x+tv)v, \quad \text{for some } t \in (0, 1). \quad (4.1)$$

From this, we derive a worst-case bound using norms:

$$\begin{aligned}\mathcal{L}_r(x)(x+\epsilon) &\leq \mathcal{L}_r(x)(x) + \|\epsilon\|_\infty \|\nabla \mathcal{L}_r(x)(x)\|_\infty + \frac{1}{2}\|\epsilon\|_\infty^2 \|\nabla^2 \mathcal{L}_r(x)(x+te)\|_\infty \\ &\leq \mathcal{L}_r(x)(x) + \|\epsilon\|_\infty \|\nabla \mathcal{L}_r(x)(x)\|_2 + \frac{\sqrt{n}}{2}\|\epsilon\|_\infty^2 \max_{\tilde{x} \in \mathbb{B}(x, \|\epsilon\|)} \|\nabla^2 \mathcal{L}_r(x)(\tilde{x})\|_2,\end{aligned} \quad (4.2)$$

where $\mathbb{B}(x, \|\epsilon\|)$ is a ball of radius $\|\epsilon\|$ centered at x . Now, to achieve a residual of zero on the training data, if ideally $\mathcal{L}_r(x) = 0$, there is no guarantee that $\mathcal{L}_r(x + \epsilon)$ will also be zero. It is evident from Eq. (4.2) that the residual at $x + \epsilon$ equals the residual at x , including their gradients and curvatures. Therefore, to equalize the residual at two points, we must ensure both zero gradient and zero curvature in the local neighborhood. If only the gradient is zero, changes in its value can be significant, leading to unpredictable residual values. To achieve optimal results, it is necessary to minimize both the gradient and the curvature simultaneously. This is the idea behind our algorithms. Moreover, based on Eq. (4.2), we can see that the curvature component has a coefficient of $\sqrt{n}$, where n is the input dimension. As the dimension increases, this coefficient grows, making it more important to minimize the curvature. In the following theorem we provide explicit connection between the local curvature, and the residual of each datapoint in the local region to justify our algorithms.

Theorem 1. Let $\mathcal{L}_r(x) = \kappa$, where x is a residual datapoint. If $\max_{\tilde{x} \in \mathbb{B}(x, \epsilon)} \|\nabla^2 \mathcal{L}_r(\tilde{x})\|_2 = \tau$, then the following inequality holds:

$$\forall x' \in \mathbb{B}(x, \epsilon), \quad \mathcal{L}_r(x') \leq \sqrt{n} \tau \epsilon^2 + 2\kappa. \quad (4.3)$$

Proof. See $\square$

Remark 2. Theorem 1 establishes that by employing the RCuPINN and CuPINN methods, we can achieve not only a reduction in the residuals at specific residual data points but also extend this reduction to the surrounding neighborhood. This is accomplished by minimizing both the curvature in the local neighborhood and the residuals at the residual data points themselves. As a result, the method provides a form of certification that residuals in the entire ϵ -neighborhood around each residual point are also minimized. More concretely, the theorem shows that the residual loss at any point x' in the neighborhood $\mathbb{B}(x, \epsilon)$ is bounded by the sum of two terms: a curvature-dependent term $\frac{\sqrt{n}}{2} \tau \epsilon^2$ and a residual term 2κ , where τ is the maximum curvature in the neighborhood and κ is the residual at the central point x . This implies that reducing the curvature (as measured by the Hessian) directly helps to keep the residuals low not just at the residual points, but across their local neighborhoods as well. Importantly, the coefficient $\sqrt{n}$ in the curvature term reflects the influence of dimensionality. As the input dimension n increases, this coefficient grows, amplifying the importance of curvature minimization in higher dimensions. In other words, the theorem underscores that as we deal with higher-dimensional problems, controlling the curvature becomes crucial for ensuring that residuals remain small throughout the neighborhood.

5. Experiments

In this section, we present a series of comprehensive experiments to demonstrate the effectiveness of our methods across a range of diverse problems. We begin by outlining the experimental configurations, including specific settings and parameters. Following this, we compare our methods with standard PINN and the state-of-the-art GPINN to highlight the superior performance of our approach. Not only do our methods achieve better approximation for u , but they also significantly outperform them in minimizing the residual loss. We specifically focus on problems in low-data scenarios and extend our analysis to situations with abundant data, showcasing the efficacy of our methods even when data availability is limited. While this advantage is less critical in low-dimensional settings – where residual data can be sampled comprehensively – it becomes crucial in higher-dimensional spaces where exhaustive sampling is infeasible. Thus, a method that generalizes well with fewer data points is essential. Our experimental evaluation proceeds from low-dimensional problems to increasingly high-dimensional ones, showcasing the scalability and robustness of our methods in handling complex, high-dimensional scenarios.

5.1. Experimental setups

Equations. Following previous works on PINNs, we provide a comprehensive set of Equations to evaluate our methods. We conduct experiments on: Damped Harmonic Oscillator Equation, 1D Poisson Equation, Diffusion Equation, Poisson Equation in 2D, 3D, 4D, Kuramoto–Sivashinsky Equation, Schrödinger Equation, and Burger's Equation. For all these Equations, we create test sets that cover the entire domain, and all plots are based on test sets that the models have not seen during training. Detailed information regarding these Equations can be found in Table 1. All experiments were conducted on Google Colab.

5.2. Damped harmonic oscillator

The ordinary differential equation (ODE) given by:

$$m \frac{d^2 u}{dt^2} + \mu \frac{du}{dt} + \kappa u = 0,$$

With $\delta = \frac{\mu}{2m}$, and $w_0 = \sqrt{\frac{\kappa}{m}}$, and $\delta < w_0$. The initial conditions are:

$$\begin{aligned} u(t=0) &= 1, \\ \frac{du}{dt}(t=0) &= 0. \end{aligned}$$

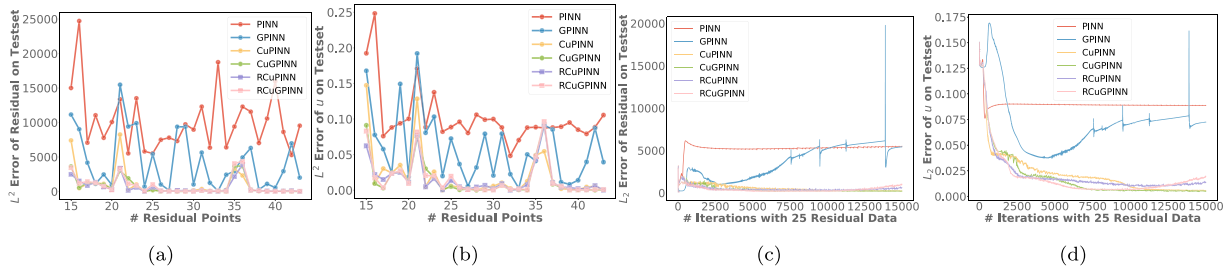


Fig. 1. Results for the Damped Harmonic Equation: (a) Residual error using varying amounts of training data. (b) Approximation error of u . (c) Residual error per epoch with 25 residual points. (d) Approximation error of u with 25 residual points.

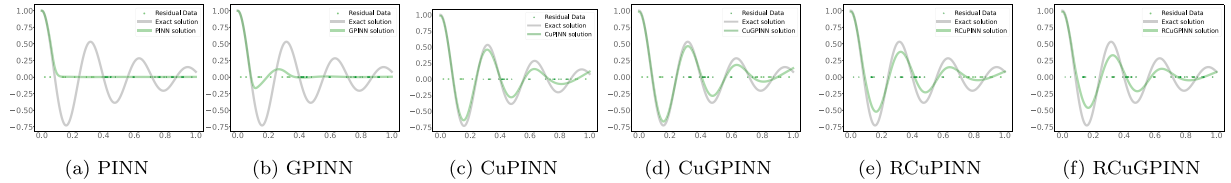


Fig. 2. Solutions for the Damped Harmonic Oscillator Using Different Methods with 25 Residual Data Points: Green dots represent the residual data points. The green line indicates the network's estimation, while the gray line shows the exact solution. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

For this particular case, the exact solution is known and given by:

$$u(t) = e^{-\delta t} (2A \cos(\phi + \omega t)), \quad \text{with} \quad \omega = \sqrt{\omega_0^2 - \delta^2}.$$

For this problem, we set $m = 1$, $\mu = 4$, and $\kappa = 400$. This configuration emphasizes the efficiency of our methods by relying solely on a single initial point and no boundary conditions, focusing entirely on residual data points for training. In Fig. 1(a), our algorithms demonstrate a steady decline in residual loss, in sharp contrast to the oscillatory behavior observed in PINN and GPINN. This consistent pattern indicates the superior training stability of our methods. While GPINN struggles significantly as the number of residual points increases from 15 to 44, our algorithms maintain stable and low residual errors, typically ranging from 20 to 500, whereas GPINN's errors are often between 500 and 5000. Fig. 1(b) illustrates the network output loss, where our methods clearly outperform both GPINN and PINN. Our algorithms achieve an approximation error of 1×10^{-3} in most cases, whereas GPINN reaches only 1×10^{-2} , confirming the improved accuracy and generalization of our approach. In Fig. 1(c), the residual loss during training with 25 residual points is presented. Our algorithms consistently avoid poor local minima and converge to better solutions, demonstrating robustness in the optimization process. Finally, Fig. 2 compares the exact ODE solution with predictions using 25 residual points. Both PINN and GPINN fail to capture the solution accurately, primarily due to insufficient residual coverage. In contrast, our algorithms effectively approximate the correct solution, showcasing the advantage of minimizing curvature—even in small neighborhoods around residual points.

5.3. 1D Poisson equation

In this section, we consider the 1D Poisson equation given by:

$$-\frac{d^2 u(x)}{dx^2} = \sum_{i=1}^4 i \sin(ix) + 8 \sin(8x), \quad x \in [0, \pi],$$

with Dirichlet boundary conditions:

$$u(0) = 0, \quad u(\pi) = \pi.$$

The exact solution for this problem is:

$$u(x) = x + \sum_{i=1}^4 \frac{\sin(ix)}{i} + \frac{\sin(8x)}{8}.$$

To satisfy the boundary conditions exactly, we define the surrogate solution as:

$$\hat{u}(x) = x(\pi - x)N(x) + x,$$

where $N(x)$ is a neural network. This formulation guarantees that the boundary conditions are met for any output of the network. We conduct experiments starting with 10 residual points and gradually increase this number up to 30 to study the effect of residual

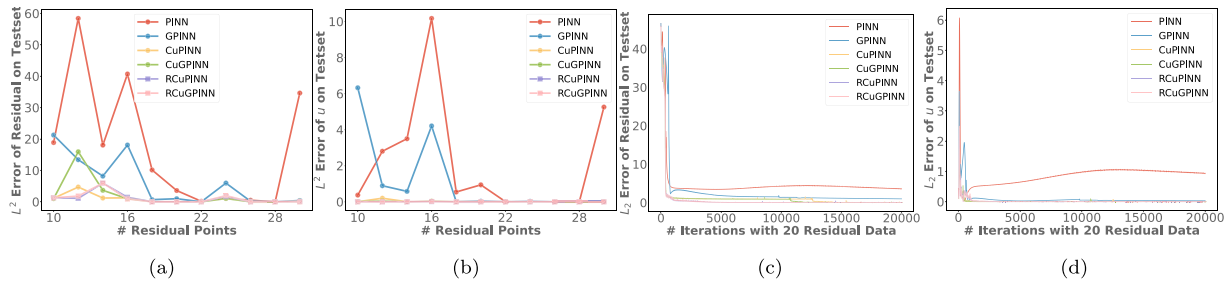


Fig. 3. Results for the 1D Poisson Equation: (a) Residual error using varying amounts of training data. (b) Approximation error of u . (c) Residual error per epoch with 20 residual points. (d) Approximation error of u with 20 residual points.

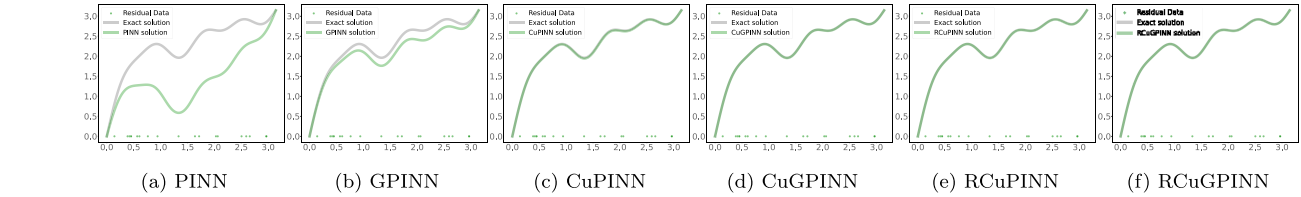


Fig. 4. Solution for the 1D Poisson Equation Using Different Methods with 20 Residual Data Points: Green dots represent the residual data points. The green line indicates the network's estimation, while the gray line shows the exact solution. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

sampling on training and accuracy. Fig. 3 summarizes the results for the 1D Poisson equation. The first row (Figs. 3(a)–3(d)) illustrates both residual and approximation errors under varying data regimes. Figs. 3(a) and 3(b) show overall residual and approximation errors as the number of residual points increases, while Figs. 3(c) and 3(d) focus on the training dynamics with only 20 residual points. Our method consistently achieves lower errors across all settings. Specifically, for the approximation error, our method typically stays in the range of 10^{-5} , whereas GPINN remains around 10^{-4} , and PINN performs significantly worse. For residual error, our method remains close to 10^{-4} , while GPINN fluctuates around 10^{-1} , often one to two orders of magnitude higher. Fig. 4 displays the predicted solutions using only 20 residual points. Unlike PINN and GPINN, which visibly deviate from the exact solution, our method provides an accurate match, clearly demonstrating better generalization.

5.4. Diffusion equation

The partial differential equation (PDE) given by:

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2} + R(x, t), \quad x \in [-\pi, \pi], \quad 0 < t < T,$$

where u is the solute concentration, $D = 1$ represents the diffusion coefficient, and R is the chemical reaction as

$$R(x, t) = e^{-t} \left[\frac{3}{2} \sin(2x) + \frac{8}{3} \sin(3x) + \frac{15}{4} \sin(4x) + \frac{63}{8} \sin(8x) \right].$$

The initial and boundary conditions are as follows:

$$u(x, 0) = \sum_{i=1}^4 \frac{\sin(ix)}{i} + \frac{\sin(8x)}{8},$$

$$u(-\pi, t) = u(\pi, t) = 0,$$

which yields the analytic solution for u as:

$$u(x, t) = e^{-t} \left[\sum_{i=1}^4 \frac{\sin(ix)}{i} + \frac{\sin(8x)}{8} \right].$$

This example is more generalized than the previous one, extending the problem to a 1D domain. Here, we set $T = 1$ and use 60 training points for the initial and boundary conditions. While the earlier example focused on a smaller region, this one broadens the spatial domain by extending the problem in the x -dimension, making it more complex. To handle the larger training region, we use a finite step of 0.02. As shown in Fig. 5(a), our algorithms outperform the other methods in residual error and demonstrate greater stability. Even when GPINN approaches the performance of our methods, its residual error remains around 10^{-1} , whereas our methods achieve errors as low as 10^{-3} . Similar trends are observed for the approximation error. In Fig. 5(b), GPINN's approximation

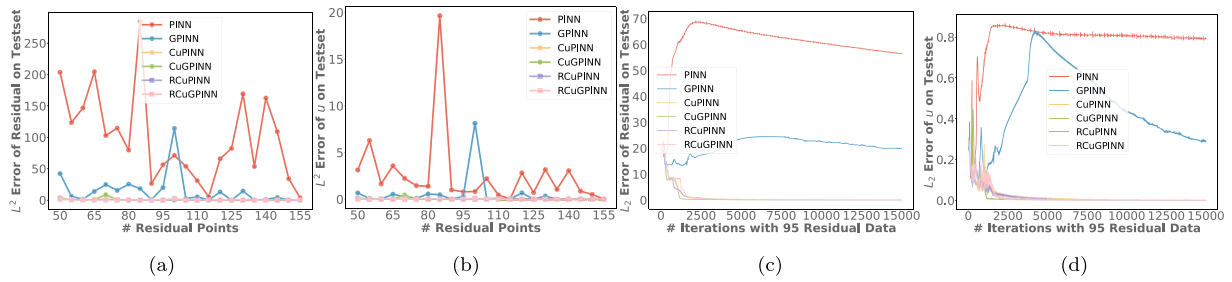


Fig. 5. Results for the Diffusion Equation: (a) Residual error using varying amounts of training data. (b) Approximation error of u . (c) Residual error per epoch with 95 residual points. (d) Approximation error of u with 95 residual points.

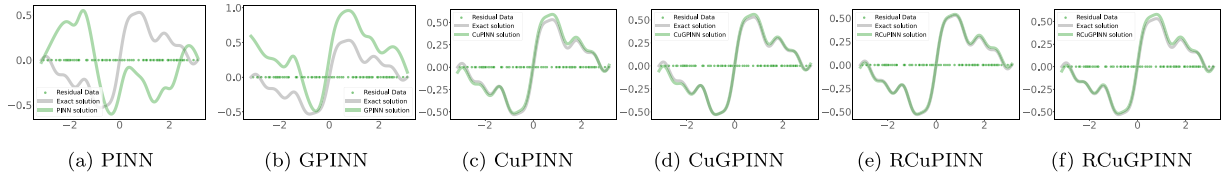


Fig. 6. Solution for the Diffusion Equation Using Different Methods with 95 Residual Data Points: Green dots represent the residual data points. The green line indicates the network's estimation, while the gray line shows the exact solution. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

error is between 10^{-3} and 10^{-4} , while our methods achieve errors on the order of 1×10^{-5} . Figs. 5(c) and 5(d) show that while GPINN and PINN manage to minimize the u error and deliver acceptable solutions, their residuals continue to grow. In contrast, our methods consistently achieve both low approximation errors and low residuals. Figs. 6(a) to 6(f) compare the exact solution with the results from different methods, clearly showing that our algorithms outperform the others. Notably, these results are consistent across varying numbers of residual data points, with our methods maintaining low residuals and error terms in all cases.

5.5. Poisson equation

In this section, we will consider the 2D Poisson equation given by:

$$\frac{\partial^2 u(x, y)}{\partial x^2} + \frac{\partial^2 u(x, y)}{\partial y^2} = -2 \sin(x) \sin(y), \quad (x, y) \in [a, b]^2,$$

with boundary condition:

$$u(x, y) = \sin(x) \sin(y), \quad (x, y) \in \partial[a, b]^2.$$

The exact solution for this 2D problem is $\sin(x) \sin(y)$, with the domain extending from $a = -5$ to $b = 5$. Due to the vast size of the domain and the limited availability of boundary values, this problem presents a significant challenge, especially for methods like PINN and GPINN, which struggle to solve it accurately. In this experiment, we use 400 boundary points and vary the number of residual data points from 50 to 150 to showcase the consistency and superior performance of our methods. Given the large domain, a finite step size of 0.2 was selected. As illustrated in Fig. 7(a), our methods outperform GPINN and PINN in residual error across all tested residual points. While GPINN approaches the performance of our algorithms after 80 residual points, its error remains on the order of 10^{-2} , whereas our methods achieve errors on the order of 10^{-3} . A similar trend is observed in the approximation of the u -function. Although GPINN narrows the gap, the error for our methods is on the order of 10^{-4} , compared to 10^{-3} for GPINN. Notably, our methods consistently reduce both residual and approximation errors for u , a trend not observed with the other methods. While PINN and GPINN exhibit fluctuations, our methods show a stable, downward trend in error reduction. Figs. 7(c) and 7(d) show the residual and approximation errors of u for 120 residual points, where our methods clearly escape local minima and provide more accurate solutions. Additionally, Fig. 8 presents a heatmap of the equation, further demonstrating that our algorithms learn significantly better solutions compared to the other methods. These results highlight the superior performance of our methods in minimizing errors and accurately capturing the solution to the Poisson equation.

5.6. Kuramoto Sivashinsky

The nonlinear PDE is given by:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + \alpha \frac{\partial^2 u}{\partial x^2} + \beta \frac{\partial^3 u}{\partial x^3} + \gamma \frac{\partial^4 u}{\partial x^4} = 0, \quad x \in [a, b], \quad 0 < t < T,$$

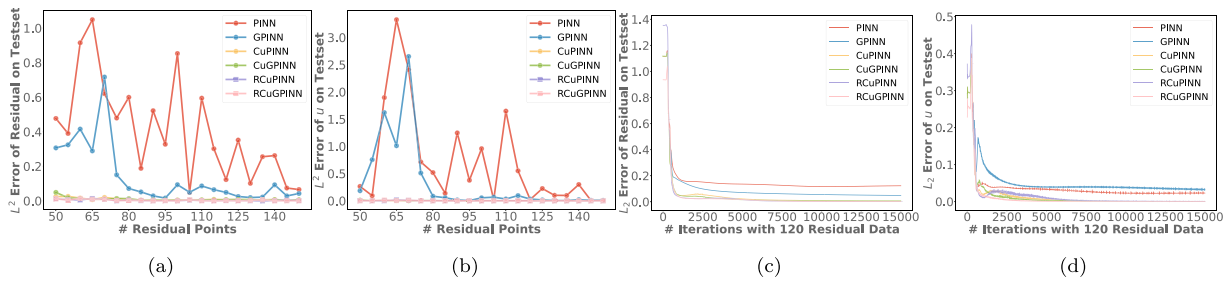


Fig. 7. Results for the Poisson Equation: (a) Residual error using varying amounts of training data. (b) Approximation error of u . (c) Residual error per epoch with 120 residual points. (d) Approximation error of u with 120 residual points.

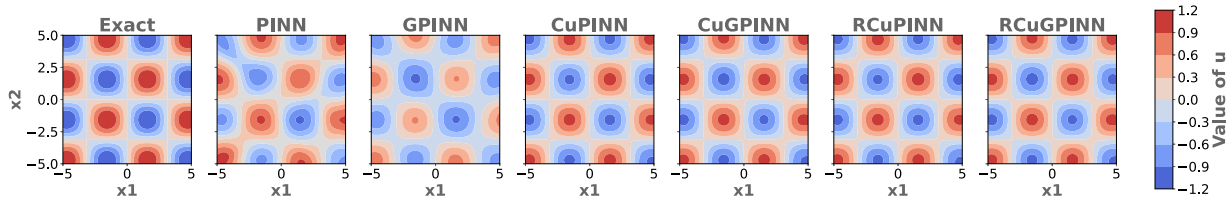


Fig. 8. Exact Solution for the Poisson Equation Using Different Methods with 120 Residual Data Points.

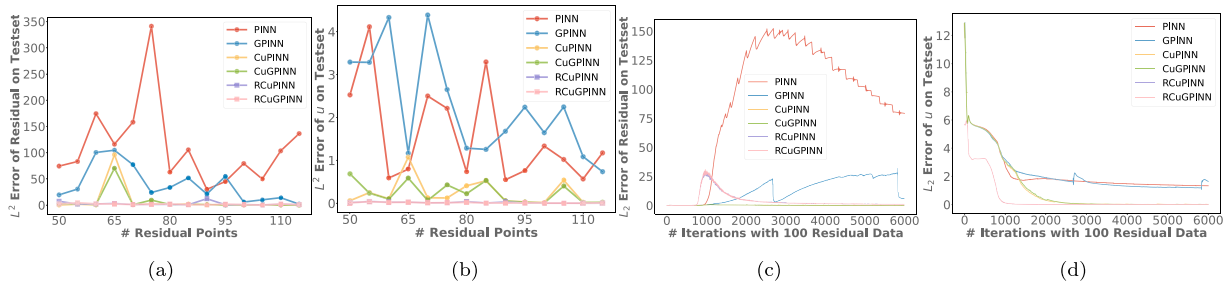


Fig. 9. Results for the Kuramoto Sivashinsky Equation at $T = 1$: (a) Residual error using varying amounts of training data. (b) Approximation error of u . (c) Residual error per epoch with 100 residual points. (d) Approximation error of u with 100 residual points.

with boundary, and initial conditions:

$$\begin{aligned} u(x, 0) &= u_{\text{exact}}(x, 0), & u(a, t) &= u_{\text{exact}}(a, t), \\ u(b, t) &= u_{\text{exact}}(b, t), & u_x(a, t) &= \frac{\partial u_{\text{exact}}}{\partial x}(a, t), \\ u_x(b, t) &= \frac{\partial u_{\text{exact}}}{\partial x}(b, t), \end{aligned}$$

where for $\beta = 0$:

$$u_{\text{exact}}(x, t) = -\frac{1}{\kappa} + \frac{60}{19} \kappa (-38\gamma\kappa^2 + \alpha) \tanh(\theta) + 120\gamma\kappa^3 \tanh^3(\theta).$$

Here, $\theta = \kappa x + t$ and $\kappa = \left(\frac{1}{2}\right) \sqrt{\frac{11\alpha}{19\gamma}}$, $\alpha = 2$, $\beta = 0$, and $\gamma = 1$, with $a = -10$ and $b = +10$. This example exhibits unique properties, including higher-order derivatives in both the boundary condition and the residual terms, along with a wider space in the x -dimension. We demonstrate the stability of our algorithms, despite the fact that GPINN and PINN do not perform well in this scenario. Notably, we use two different settings for this problem:

1. First Setting: We set $T = 1$ and sample data points within this region. The number of boundary and initial points is 50, and the time step is set to 0.01, as shown in Table 1.

2. Second Setting: We extend T to 10 to solve the problem over a larger space and evaluate performance, as the function behavior changes significantly. For this setup, we reduce the number of boundary and initial points to 20 to test the generalization power of our methods. The time step is set to 0.1 to account for the increased spatial domain.

Results for $T = 1$. As shown in Fig. 9, both PINN and GPINN exhibit significant fluctuations, with large variations in their u -loss and residual loss. In contrast, our methods demonstrate far more stable behavior, consistently achieving lower u -loss and residual loss compared to PINN and GPINN across all cases. Our methods provide better solutions, with approximation error around 1×10^{-2} ,

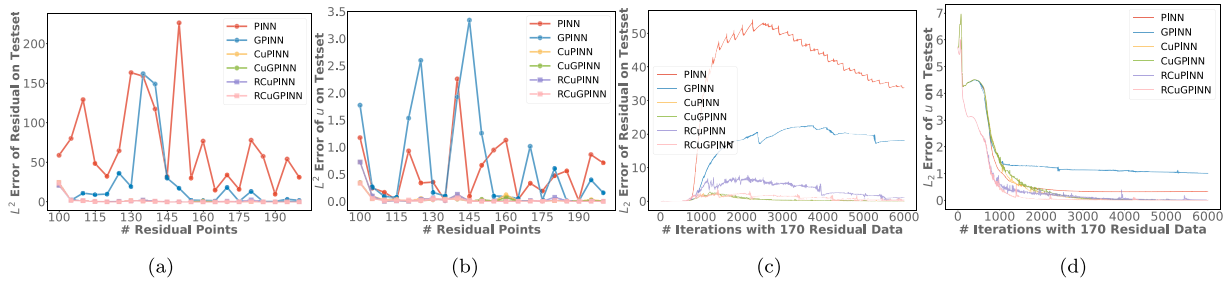


Fig. 10. Results for the Kuramoto Sivashinsky Equation at $T = 10$: (a) Residual error using varying amounts of training data. (b) Approximation error of u . (c) Residual error per epoch with 170 residual points. (d) Approximation error of u with 170 residual points. We do not provide PINN results in some figures since its error was too large, making it difficult to visualize the errors of other methods.

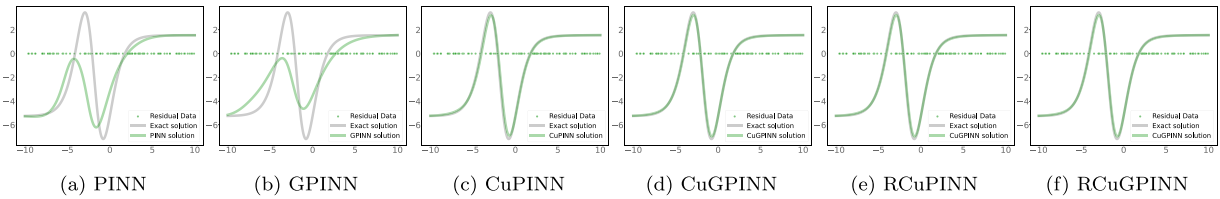


Fig. 11. Exact Solution for the Kuramoto Sivashinsky with $T = 1$ Using Different Methods with 100 Residual Data Points: Green dots represent the residual data points. The green line indicates the network's estimation, while the gray line shows the exact solution. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

compared to errors around 1 for other methods. Moreover, our methods achieve significantly lower residuals, which PINN and GPINN fail to reach. It is important to note that the residual loss for CuPINN and CuGPINN with 85 residual data points is on the order of 1×10^{-1} , while PINN and GPINN show significantly higher residuals. From Fig. 9(c), and 9(d), it is evident that although GPINN and PINN minimize the approximation error, they struggle to effectively minimize the residual loss across the entire domain, often getting trapped in poor local minima. The exact solution and solutions from the different methods are illustrated in Fig. 11, where the superiority of our methods is clearly visible.

Results for $T = 10$. For the second setting, with $T = 10$, we observe expanded function variability over the extended domain. The results are depicted in Fig. 10. Our methods significantly outperform GPINN and PINN, which continue to exhibit fluctuating behavior in this extended setting. Our methods maintain stable performance, successfully minimizing both residual and approximation loss something GPINN and PINN fail to do. Figs. 10(c) and 10(d) show that while GPINN and PINN reduce the approximation error, they struggle to minimize the residual loss effectively.

5.7. Schrödinger equation

The non-linear partial differential equation (PDE) given by [39]:

$$i \frac{\partial \Psi}{\partial t} + \alpha \frac{\partial^2 \Psi}{\partial x^2} + \beta |\Psi|^2 \Psi = 0 \quad x \in [a, b], \quad t \in [0, T],$$

with boundary, and initial condition:

$$\Psi(x, 0) = \Psi_{\text{exact}}(x, 0), \quad x \in [a, b],$$

$$\Psi(a, t) = \Psi_{\text{exact}}(a, t), \quad t \in [0, T],$$

$$\Psi(b, t) = \Psi_{\text{exact}}(b, t), \quad t \in [0, T],$$

where:

$$\Psi_{\text{exact}}(x, t) = \text{sech}(x - 4t - 10) \exp(-i(2x - 3t - 20)) \\ + \text{sech}(x + 4t + 10) \exp(i(2x + 3t + 20)).$$

We set $\alpha = -1$, $\beta = -2$, $a = -21$, $b = +21$, and $T = \pi/2$. This problem is particularly challenging due to the large domain, where the function is nearly zero across most of the area. Consequently, the network might tend to learn a trivial zero solution. Additionally, the problem involves two solitons moving apart, further increasing its complexity. Physics-Informed Neural Networks (PINNs), even with substantial data, struggle to accurately solve this problem. The challenge of the two-soliton Schrödinger equation arises from the nonlinear interactions between the solitons, which lead to complex dynamics during collisions. Although solitons preserve their individual shapes, their interactions cause intricate phase shifts and velocity changes, making the problem especially difficult. This problem poses a greater challenge than typical PINNs examples, mainly because of the presence of two solitons. Each

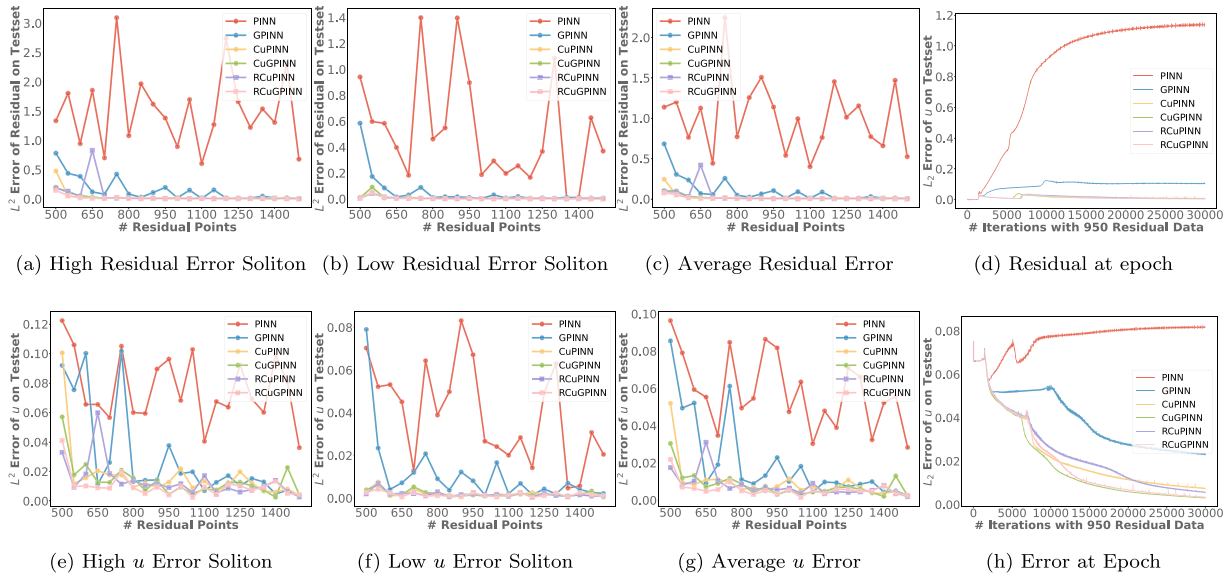


Fig. 12. Results for the Schrödinger Equation: In this experiment, we analyze two solitons: the *high error soliton* with higher loss and the *low error soliton* with lower loss. (a) Residual error for the high error soliton using varying amounts of training data. (b) Residual error for the low error soliton. (c) Residual error for the average error. (d) Residual error per epoch with 950 residual points. (e) Approximation error for the high error soliton. (f) Approximation error for the low error soliton. (g) Approximation error for the average error. (h) Approximation error per epoch with 950 residual points.

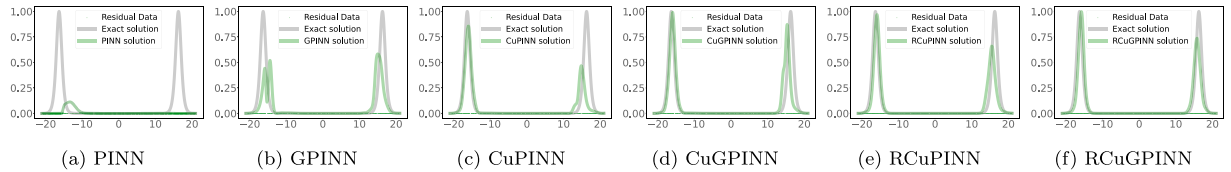


Fig. 13. Solutions for the Schrödinger Equation Using Different Methods with 950 Residual Data Points at $T = \frac{\pi}{2}$: Green dots represent the residual data points. The green line indicates the network's estimation, while the gray line shows the exact solution. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

soliton exhibits sharp behavior, being nearly zero across most of the domain except for small regions of non-zero values. This sparsity complicates loss calculation, potentially creating a false sense of approximation accuracy as the model may fit the large zero-valued areas without capturing the intricate dynamics of the solitons. To address this, we evaluate the test set in regions where the soliton values exceed 5×10^{-2} , focusing on the meaningful dynamics. For a more detailed evaluation, we report the test loss for each soliton individually as well as the average loss across both solitons. Notably, the algorithms tend to learn the solitons asymmetrically, focusing on one soliton first, then the other, resulting in different loss values for each soliton. We classify the solitons based on their residual and approximation losses, labeling the soliton with the higher loss as the *high error soliton* and the other as the *low error soliton*. Figs. 12(a) to 12(c) illustrate the residual loss for both solitons. Our algorithms consistently performs significantly better than PINN, showing a clear advantage. While GPINN achieves residual error on the order of 1×10^{-1} , our methods reduces the error to 1×10^{-3} , demonstrating superior residual learning and a better ability to capture the sharp dynamics of each soliton. In Fig. 12(d), which displays the residual on the test set with 950 data points, both PINN and GPINN diverge after initially starting with near-zero residuals. In contrast, our algorithms converges to a much lower residual, indicating more accurate residual learning. Figs. 12(e) to 12(g) further support these findings for function approximation. Our algorithm consistently exhibits greater stability compared to both GPINN and PINN. Although GPINN sometimes achieves slightly lower approximation error, its residual error remains higher, as shown in Fig. 13. In this figure, GPINN's solution includes sharp, incorrect drops due to poor residual learning. Finally, in Fig. 12(h), we display the approximation loss, where it is clear that our algorithm converges to much better local minima than the competing methods.

5.8. Burger's equation

The non-linear 1D Burgers equation:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in [-1, 1], \quad t \in [0, 1],$$

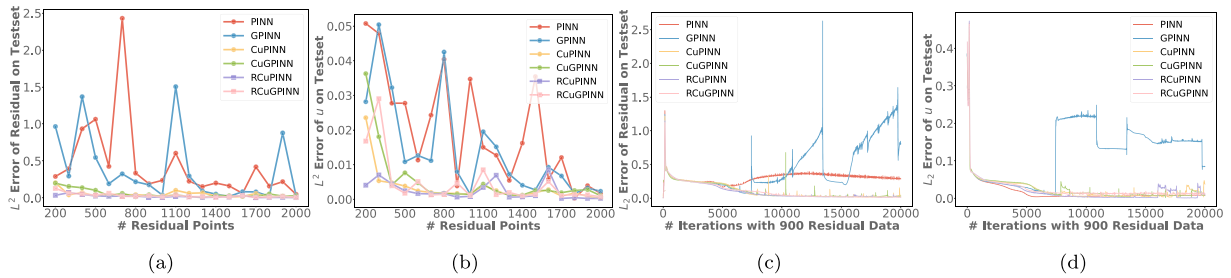


Fig. 14. Results for the Burger's Equation: (a) Residual error using varying amounts of training data. (b) Approximation error of u . (c) Residual error per epoch with 900 residual points. (d) Approximation error of u with 900 residual points. We do not provide PINN results in some figures since its error was too large, making it difficult to visualize the errors of other methods.

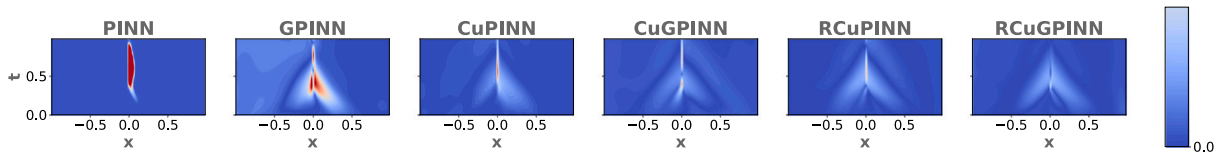


Fig. 15. Heatmap of the Difference Between the Reference Solution and Outputs of Different Methods for Burger's Equation Using 900 Residual Data Points. The heatmaps highlight the accuracy of each method in solving Burger's equation, with blue regions indicating close alignment with the reference solution and red regions showing areas of significant deviation. Our method demonstrates a closer match to the reference solution compared to the other methods.

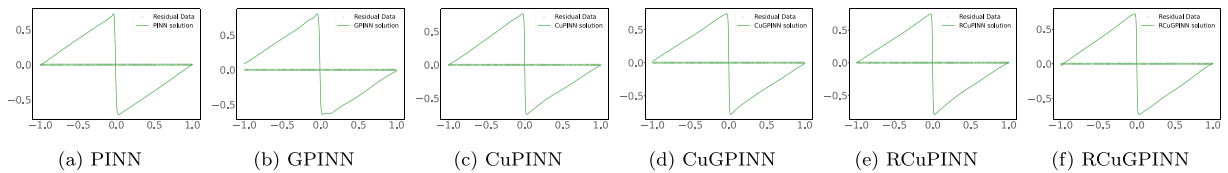


Fig. 16. Solutions for the Burger's Equation with $T = 1$ Using Different Methods with 900 Residual Data Points: Green dots represent the residual data points. The green line indicates the network's estimation.

with the initial and boundary conditions

$$u(x, 0) = -\sin(\pi x), \quad u(-1, t) = u(1, t) = 0,$$

with $\nu = 0.01/\pi$. Burger's equation presents significant challenges that many numerical algorithms struggle to address. In this section, we explore this equation and conduct a series of experiments similar to those previously described. We perform comprehensive experiments across a broad range of residual data points, varying from 200 to 2000, to assess the performance of our method. Additionally, we generate a test set that covers the entire problem domain using the Finite Volume Method (FVM). From this dataset, we randomly selected 10,000 data points for evaluation. As shown in Fig. 14(a), our algorithms demonstrates significantly more stable residual loss compared to both PINN and GPINN, with notably fewer fluctuations. Similarly, Fig. 14(b) presents the approximation loss, revealing a consistent trend with the residual loss. Our algorithms consistently outperforms PINN and GPINN in terms of approximation accuracy. Additionally, Figs. 14(c) and 14(d) illustrate the residual and approximation loss during training with 900 interior data points. It is evident that, unlike other methods, which struggle to escape local minima, our algorithm is more successful in overcoming these challenges, particularly in comparison to GPINN. Fig. 15 highlights the differences between the reference and estimated values across the algorithms. The blue regions represent areas where the estimated solution aligns closely with the reference solution, while the red regions indicate areas of significant deviation. As shown, our methods achieves far better results compared to both PINN and GPINN. It is well known that Burger's equation (in this special case) develops discontinuities around zero after a certain time step. As demonstrated in Fig. 16, both PINN and GPINN exhibit substantial deviations near the discontinuities. In contrast, our algorithms provide a much closer approximation, handling these discontinuities more accurately.

6. Further analysis & discussion

6.1. Higher dimensions & random directions

In this work, we propose two algorithmic strategies for curvature minimization: one based on the gradient direction and the other on a random direction. Both strategies aim to identify and reduce high-curvature regions on the residual surface, but they differ

Table 2

Performance of various algorithms across different dimensions with random direction curvature minimization. The table shows both residual loss and approximation loss for each algorithm evaluated in dimensions 2, 3, and 4.

Dimension	Method	Residual Error	Approximation Error
2	PINN	1e-4	3.25e-6
	GPINN	1.1e-4	4.1e-6
	CuPINN	6.1e-5	1.7e-7
	RCuPINN	9.52e-5	3.23e-6
	CuGPINN	7.94e-5	2.67e-6
	RCuGPINN	8.69e-4	3.03e-6
3	PINN	0.29	0.017
	GPINN	0.072	6.1e-3
	CuPINN	1e-2	4e-3
	RCuPINN	2.4e-2	2.2e-3
	CuGPINN	1.8e-2	2e-3
	RCuGPINN	3e-2	3.2e-3
4	PINN	4.99	0.24
	GPINN	2.67	0.15
	CuPINN	0.74	0.089
	RCuPINN	1.0	0.12
	CuGPINN	0.92	0.11
	RCuGPINN	1.15	0.13

in computational cost and precision. The gradient-based approach, implemented in CuPINN and CuGPINN, focuses on minimizing curvature along the gradient direction, which requires backpropagation through the network but results in more precise updates. To reduce computational overhead, we also introduce RCuPINN and RCuGPINN, which follow the same framework but replace the gradient direction with a random one. This modification enables simultaneous computation of residuals at main and random points, similar to GPINN, leading to slightly lower computational cost. Although all four methods are fundamentally similar, experiments in Table 2 demonstrate that CuPINN and CuGPINN perform slightly better than their random-direction counterparts. Nonetheless, all four variants significantly outperform baseline methods like PINN and GPINN, especially in higher-dimensional problems such as the Poisson equation from 2D to 4D. Regarding the experimental setup, for dimension 2, we used 200 boundary points and 500 residual points. For dimension 3, the setup included 300 boundary points and 1000 residual points, and for dimension 4, we used 400 boundary points and 2000 residual points. As dimensions increase, the number of residual points is scaled exponentially. This increase highlights that the performance of PINN and GPINN degrades significantly in higher dimensions. The general form of the Poisson equation we consider is:

$$\sum_{i=1}^N \frac{\partial^2 u(x)}{\partial x_i^2} = -N \prod_{i=1}^N \sin(x_i), \quad \text{where } x_i \in [a, b]^N$$

with the exact solution given by:

$$u(x) = \prod_{i=1}^N \sin(x_i)$$

It is important to note that for higher-dimensional Poisson equations (e.g., dimensions 5 and above), the solution space can become nearly zero. Consequently, the loss or relative loss values may initially be on the order of 10^{-1} , which can create a misleading impression of learning progress. Thus, our analysis focuses on dimensions where the solution space is not nearly zero.

6.2. Properties of gradient direction

In the proposed algorithm, we compute the expectation by considering a single direction. Ideally, this direction could correspond to the maximum eigenvector of the Hessian matrix, which points in the direction of the steepest curvature. However, calculating the maximum eigenvector is computationally expensive. Instead, we use the gradient direction, which is both computationally efficient and well-aligned with the eigenvector corresponding to the largest eigenvalue of the Hessian in many practical cases. Below, we demonstrate the relationship between the gradient direction and the maximum eigenvector of the Hessian. Let the loss function associated with the residual be defined as:

$$L(x) = \frac{1}{2} r(x)^2$$

where $r(x)$ represents the residual function as defined in Section 3. The gradient and Hessian of this loss function are:

$$\nabla L(x) = r(x) \nabla r(x), \quad \nabla^2 L(x) = \nabla r(x) \nabla r(x)^T + r(x) \nabla^2 r(x)$$

We are interested in the maximum eigenvalue of the Hessian $\nabla^2 L(x)$, which is given by:

$$\lambda_{\max} = \max_{\|v\|=1} v^T \nabla^2 L(x) v$$

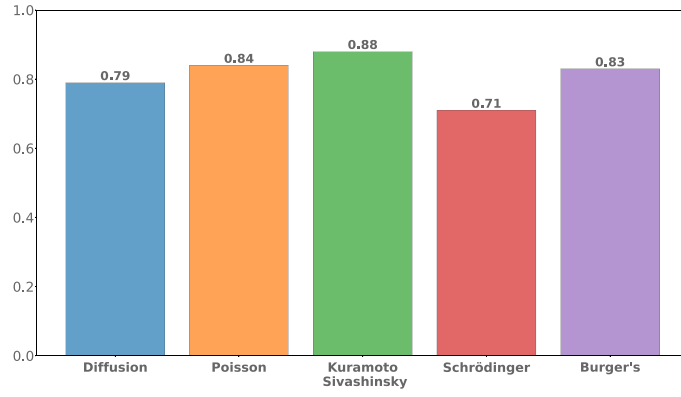


Fig. 17. The Average Cosine similarities between the eigenvectors corresponding to the maximum eigenvalue and the gradient directions across different epochs, computed for 1000 datapoints.

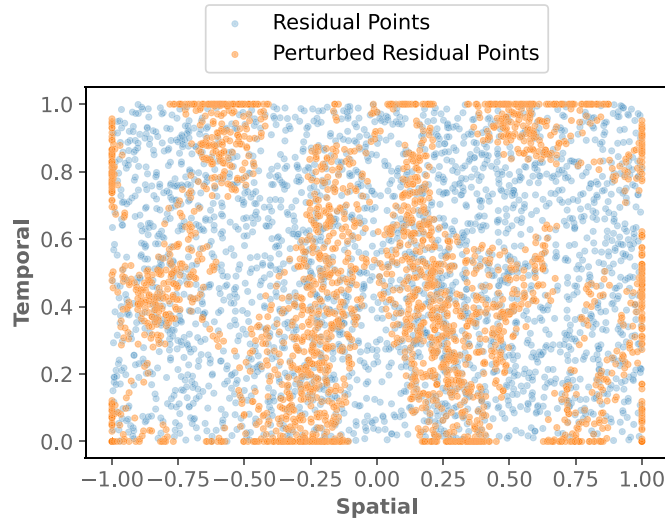


Fig. 18. Distribution of Residual and Perturbed Residual points during the training of the Burger's equation. Blue dots indicate Residual points, while orange dots represent perturbed points, which are concentrated in the discontinuity region of the equation. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Substituting the expression for the Hessian:

$$v^T \nabla^2 L(x) v = v^T \nabla r(x) \nabla r(x)^T v + r(x) v^T \nabla^2 r(x) v$$

This simplifies to:

$$v^T \nabla^2 L(x) v = \|\nabla r(x)^T v\|^2 + r(x) v^T \nabla^2 r(x) v$$

From this expression, we can see that one of the terms that contributes to maximizing this quantity is $\|\nabla r(x)^T v\|^2$. The gradient direction $\frac{\nabla r(x)}{\|\nabla r(x)\|}$ maximizes this term, which suggests that the maximum eigenvector of the Hessian is closely related to the gradient direction. Moreover, the gradient direction $\nabla r(x)$ is also the direction in which the function changes the most. This provides further justification for choosing the gradient direction as an efficient and reasonable approximation for the direction corresponding to the eigenvector of maximum eigenvalue of the Hessian. In Fig. 17, we compute the cosine similarity between the eigenvector corresponding to the maximum eigenvalue of the Hessian and the gradient direction across several problems. As shown, the cosine similarity is consistently greater than 0.70, experimentally validating the strong relationship between these two directions. It is important to note that in all experiments, even the random-direction variants of our method perform significantly better than both GPINN and PINN. This remains true even when the cosine similarity between the gradient direction and the direction of the largest eigenvector of the curvature matrix is low. These results indicate that, despite the gradient direction not always aligning well with the principal curvature direction, our algorithms – including the random-direction versions – still achieve superior performance. Additionally, in Fig. 18, we visualize the distribution of interior and perturbed interior points during the training of the Burger's

Table 3

Comparison of time, residual loss, and approximation loss across various methods in the high-data regime for different equations.

Equations	Metrics	Algorithms					
		PINN	GPINN	CuPINN	CuGPINN	RCuPINN	RCuGPINN
Damped Harmonic	Speed (It/s)	93	65	40	40	62	62
	Time (s)	160	240	370	370	250	250
	Residual loss	30.94	11.72	0.21	1.36	0.35	1.10
	Approx. loss	1.4e−4	4.8e−4	3.37e−6	1.4e−5	5.63e−6	9.0e−6
Diffusion	Speed (It/s)	75	45	25	25	46	46
	Time (s)	250	330	540	540	335	335
	Residual loss	4.9e−3	1.6e−3	8.0e−4	8.8e−4	7.0e−4	7.3e−4
	Approx. loss	3.0e−5	3.0e−5	1.6e−6	2.5e−6	1.2e−6	1.4e−6
Poisson	Speed (It/s)	37	29	19	19	29	29
	Time (s)	420	525	855	855	540	540
	Residual loss	1.0e−4	1.1e−4	6.1e−5	7.9e−5	8.2e−5	9.6e−5
	Approx. loss	3.25e−6	4.1e−6	1.7e−7	2.6e−6	2.3e−6	3.1e−6
Kuramoto–Sivashinsky	Speed (It/s)	18	9	5	5	8	8
	Time (s)	340	690	1170	1170	700	700
	Residual loss	2.0e−2	9.0e−4	2.0e−4	2.0e−4	2.4e−4	4.0e−4
	Approx. loss	3.6e−5	7.0e−5	8.4e−6	1.0e−5	9.0e−6	3.0e−5
Schrödinger	Speed (It/s)	40	26	16	16	25	25
	Time (s)	750	1200	1900	1900	1230	1230
	Residual loss	5.7e−2	6.3e−3	5.1e−3	4.2e−3	2.3e−3	4.3e−3
	Approx. loss	1.4e−2	5.7e−3	5.4e−3	4.5e−3	3.7e−3	4.8e−3
Burger's	Speed (It/s)	70	40	30	30	45	45
	Time (s)	340	600	840	840	620	620
	Residual loss	2.0e−2	3.0e−3	2.0e−3	4.0e−3	9.0e−4	2.4e−3
	Approx. loss	1.2e−3	1.2e−3	1.0e−4	1.0e−3	5.3e−5	7.3e−4

equation using our algorithm. The blue dots represent the interior points, while the orange dots denote their perturbed counterparts. Notably, the perturbed points are located within the discontinuity regions of the equation, particularly around the value of 0. This finding underscores the ability of the gradient direction – computed without prior knowledge of the function – to effectively pinpoint areas where the model encounters challenges. Such robustness highlights the efficacy of our method in addressing complex regions of the solution space.

6.3. High data regime

Up to this point, we have evaluated the algorithms in a low-data regime, driven primarily by memory limitations and the increased computational time required for larger datasets. In higher-dimensional problems, adequate sampling becomes particularly challenging, as covering the space thoroughly requires exponentially more data points. However, in this section, we shift our focus to the high-data regime, where a significantly larger dataset is available with respect to the problem domain. This allows us to explore the behavior and performance of the algorithms under data-abundant conditions. As shown in Table 3, our methods consistently outperforms PINN and GPINN across all equations. In most cases, the residual and approximation errors of our algorithm are **an order of 10^{-1} lower** than those of the other methods. It is important to note that the computational time required for our algorithms is longer compared to other approaches. This is due to the fact that in PINN, there is no need to backpropagate through the network to compute the gradient of the residual error. In GPINN, a single backward pass is required to compute the gradient of the residual loss. In contrast, CuPINN and CuGPINN require two backward passes—one to determine the gradient direction and another to compute the directional derivative along that gradient. This introduces a trade-off between improved accuracy and increased computational cost. We illustrate this trade-off for the residual error in Fig. 19. However, in RCuPINN and RCuGPINN, the gradient along the random direction can be computed in parallel with the residuals, making their runtime comparable to that of GPINN.

6.4. Visualization of the residual surface

Our algorithm is designed to minimize the residual across the entire domain by reducing the eigenvalues of the Hessian matrix, effectively driving the curvature of the residual surface toward zero. This results in a significantly flatter residual surface. In contrast, as discussed earlier, other algorithms struggled to learn the residual effectively, leading to higher residual values. In this part, we highlight the superior performance of our algorithm, which achieves low residual error, demonstrating its ability to learn the physics of the equations. We visually illustrate this in Fig. 20, where the residual surface is shown for each algorithm. The residual values are calculated at every point and represented using heatmaps. Notably, the heatmaps for our algorithm are predominantly blue, indicating much smaller residual values compared to those produced by competing algorithms. The heatmaps in Fig. 20 clearly demonstrate the superiority of our algorithm, confirming its ability to more accurately capture the underlying physics of the problem than other methods.

Trade-off Plot: Time vs. Residual Loss

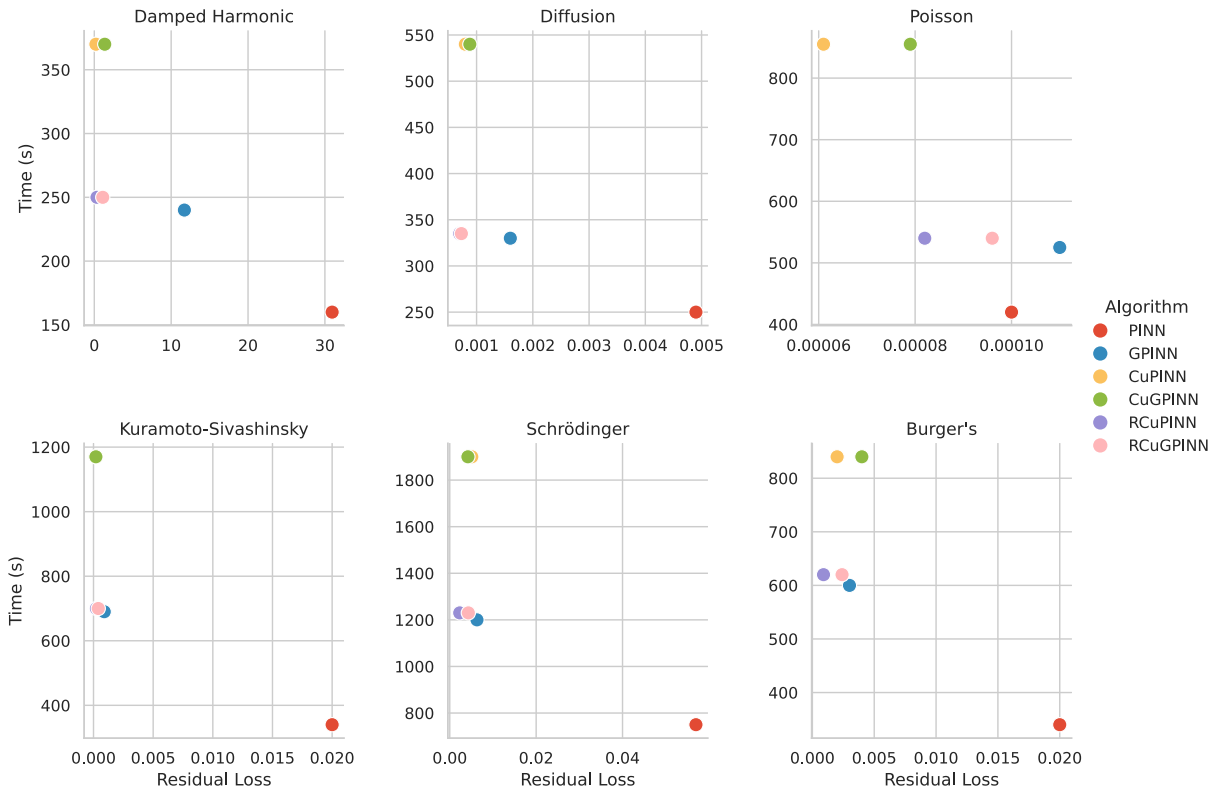


Fig. 19. Trade-off between computation time and residual loss for various algorithms across different equations. Lower values for both residual loss (X-axis) and time (Y-axis) indicate superior performance.

7. Conclusion

In this paper, we introduced four novel methods, CuPINN, CuGPINN, RCUINN and RCUGPINN, aimed at improving the accuracy and efficiency of Physics-Informed Neural Networks (PINNs) by flattening the residual surface. Our approach is built on the key insight that reducing the curvature of the residual surface, through minimizing the eigenvalues of the Hessian, leads to a smoother and more accurate solution. By enforcing this zero-curvature condition, we effectively enhance the learning process, allowing our methods to outperform existing approaches, including Gradient-Enhanced PINNs (GPINNs), especially in low-data regimes. Through both theoretical analysis and extensive experiments across a wide range of partial differential equations (PDEs), we demonstrated that our algorithms not only reduce the residual and approximation errors more effectively than other methods but also produce a significantly flatter residual surface. Our results confirm that the minimization of curvature implicitly minimizes the gradient, further enhancing solution stability and accuracy. In summary, our methods deliver superior performance in both low- and high-data regimes, and achieving higher accuracy in solving complex PDEs. Future work may focus on extending these techniques to handle even more challenging high-dimensional problems and exploring further optimizations to reduce computational overhead.

CRediT authorship contribution statement

Raouf Zare Moayedi: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Conceptualization. **Mostafa Abbaszadeh:** Writing – review & editing, Visualization, Supervision, Software, Methodology, Conceptualization. **Mehdi Dehghan:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

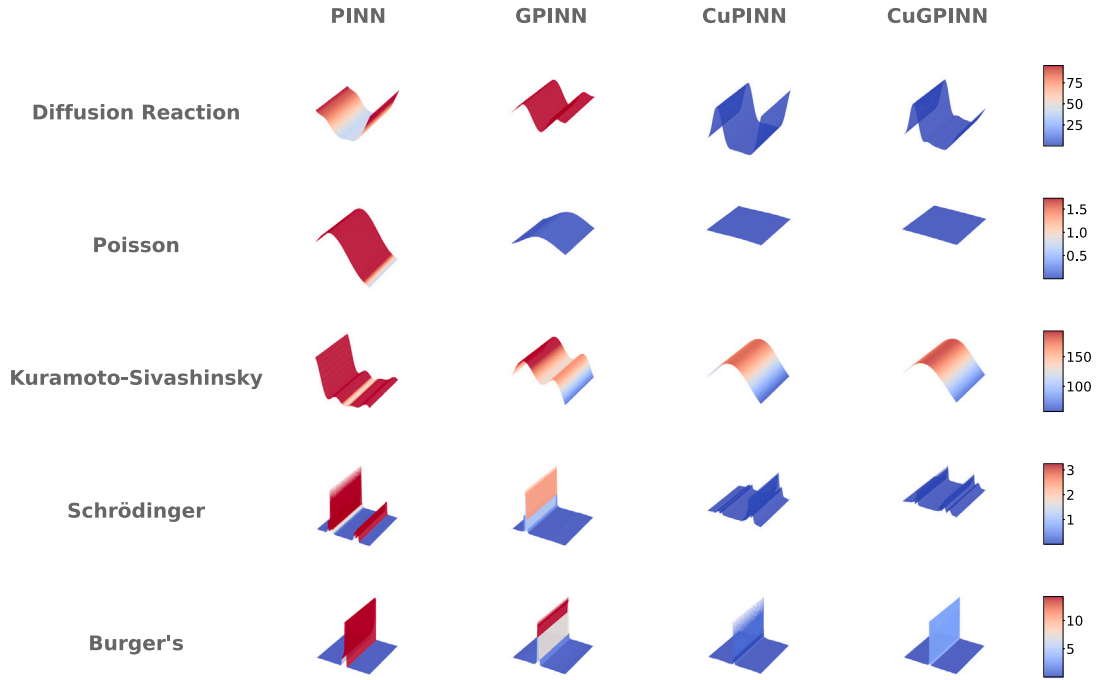


Fig. 20. The residual surface plots for different equations, obtained using various methods, are shown. For each figure, the problem's space is divided into a 100x100 grid, with residuals calculated at each point. In these visualizations, blue regions indicate low residual values, while red regions correspond to high residual values, illustrating the effectiveness of each method in minimizing residuals across the domain. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Appendix

A.1. Proof of Theorem 1

Proof. For perturbations $\pm\epsilon$, we have:

$$\mathcal{L}_r(\mathbf{x} + \epsilon) = \mathcal{L}_r(\mathbf{x}) + \nabla \mathcal{L}_r(\mathbf{x})^\top \epsilon + \frac{1}{2} \epsilon^\top \nabla^2 \mathcal{L}_r(\mathbf{x} + t_+ \epsilon) \epsilon, \quad (\text{A.1})$$

$$\mathcal{L}_r(\mathbf{x} - \epsilon) = \mathcal{L}_r(\mathbf{x}) - \nabla \mathcal{L}_r(\mathbf{x})^\top \epsilon + \frac{1}{2} \epsilon^\top \nabla^2 \mathcal{L}_r(\mathbf{x} - t_- \epsilon) \epsilon, \quad (\text{A.2})$$

where $t_+, t_- \in (0, 1)$ are different points for $+\epsilon$ and $-\epsilon$.

Summing the expansions

The gradient terms cancel out, leaving:

$$\mathcal{L}_r(\mathbf{x} + \epsilon) + \mathcal{L}_r(\mathbf{x} - \epsilon) = 2\mathcal{L}_r(\mathbf{x}) + \frac{1}{2} \epsilon^\top (\nabla^2 \mathcal{L}_r(\mathbf{x} + t_+ \epsilon) + \nabla^2 \mathcal{L}_r(\mathbf{x} - t_- \epsilon)) \epsilon. \quad (\text{A.3})$$

Bounding the Hessian terms

Using $\|\epsilon^\top \mathbf{H} \epsilon\|_\infty \leq \|\epsilon\|_\infty^2 \|\mathbf{H}\|_\infty$ and $\|\mathbf{H}\|_\infty \leq \sqrt{n} \|\mathbf{H}\|_2$, we get:

$$\epsilon^\top \nabla^2 \mathcal{L}_r(\mathbf{x} + t_+ \epsilon) \epsilon \leq \|\epsilon\|_\infty^2 \sqrt{n} \tau, \quad (\text{A.4})$$

$$\epsilon^\top \nabla^2 \mathcal{L}_r(\mathbf{x} - t_- \epsilon) \epsilon \leq \|\epsilon\|_\infty^2 \sqrt{n} \tau, \quad (\text{A.5})$$

where $\tau = \max_{\bar{\mathbf{x}} \in \mathbb{B}(\mathbf{x}, \epsilon)} \|\nabla^2 \mathcal{L}_r(\bar{\mathbf{x}})\|_2$.

Worst-case sum of Hessians

The worst case occurs when both Hessian terms attain their maximum τ :

$$\frac{1}{2} (\epsilon^\top \nabla^2 \mathcal{L}_r(\mathbf{x} + t_+ \epsilon) \epsilon + \epsilon^\top \nabla^2 \mathcal{L}_r(\mathbf{x} - t_- \epsilon) \epsilon) \leq \sqrt{n} \|\epsilon\|_\infty^2 \tau. \quad (\text{A.6})$$

This leads to:

$$\mathcal{L}_r(\mathbf{x} + \epsilon) + \mathcal{L}_r(\mathbf{x} - \epsilon) \leq 2\mathcal{L}_r(\mathbf{x}) + \sqrt{n} \|\epsilon\|_\infty^2 \tau. \quad (\text{A.7})$$

Isolating $\mathcal{L}_r(\mathbf{x} + \epsilon)$

Since $\mathcal{L}_r \geq 0$, we know:

$$\mathcal{L}_r(\mathbf{x} + \epsilon) \leq \mathcal{L}_r(\mathbf{x} + \epsilon) + \underbrace{\mathcal{L}_r(\mathbf{x} - \epsilon)}_{\geq 0} \leq 2\mathcal{L}_r(\mathbf{x}) + \sqrt{n} \|\epsilon\|_\infty^2 \tau. \quad (\text{A.8})$$

Thus, the final bound becomes:

$$\mathcal{L}_r(\mathbf{x} + \epsilon) \leq 2\mathcal{L}_r(\mathbf{x}) + \sqrt{n} \|\epsilon\|_\infty^2 \tau. \quad \square \quad (\text{A.9})$$

Data availability

No data was used for the research described in the article.

References

- [1] Kurt Hornik, Maxwell Stinchcombe, Halbert White, Multilayer feedforward networks are universal approximators, *Neural Netw.* 2 (5) (1989) 359–366.
- [2] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [3] Jeremy Yu, Lu Lu, Xuhui Meng, George Em Karniadakis, Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems, *Comput. Methods Appl. Mech. Engrg.* 393 (2022) 114823.
- [4] Tao Luo, Haizhao Yang, Two-layer neural networks for partial differential equations: Optimization and generalization theory, 2020.
- [5] Zheyuan Hu, Ameya D. Jagtap, George Em Karniadakis, Kenji Kawaguchi, When do extended physics-informed neural networks (XPINNs) improve generalization? *SIAM J. Sci. Comput.* 44 (5) (2022) A3158–A3182.
- [6] Jianfeng Lu, Yulong Lu, Min Wang, A priori generalization analysis of the deep ritz method for solving high dimensional elliptic equations, 2021.
- [7] Siddhartha Mishra, Roberto Molinaro, Estimates on the generalization error of physics informed neural networks (PINNs) for approximating PDEs, 2023.
- [8] Yeonjong Shin, On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type PDEs, *Commun. Comput. Phys.* 28 (5) (2020) 2042–2074.
- [9] Guofei Pang, Lu Lu, George Em Karniadakis, fPINNs: Fractional physics-informed neural networks, *SIAM J. Sci. Comput.* 41 (4) (2019) A2603–A2626.
- [10] Lu Lu, Xuhui Meng, Zhiping Mao, George Em Karniadakis, DeepXDE: A deep learning library for solving differential equations, *SIAM Rev.* 63 (1) (2021) 208–228.
- [11] Maziar Raissi, George Em Karniadakis, Hidden physics models: Machine learning of nonlinear partial differential equations, *J. Comput. Phys.* 357 (2018) 125–141.
- [12] Ehsan Haghighat, Maziar Raissi, Adrian Moure, Hector Gomez, Ruben Juanes, A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics, *Comput. Methods Appl. Mech. Engrg.* 379 (2021) 113741.
- [13] Yibo Yang, Paris Perdikaris, Adversarial uncertainty quantification in physics-informed neural networks, *J. Comput. Phys.* 394 (2019) 136–152.
- [14] Shengze Cai, Zhiping Mao, Zhicheng Wang, Minglang Yin, George Em Karniadakis, Physics-informed neural networks (PINNs) for fluid mechanics: a review, *Acta Mech. Sin.* 37 (2021) 1727–1738.
- [15] Nima Mohammadi, Mostafa Abbaszadeh, Mehdi Dehghan, Clemens Heitzinger, Parameter identification of shallow water waves using the generalized equal width equation and physics-informed neural networks: a conservative approximation scheme, *Nonlinear Dyn.* 113 (7) (2025) 6491–6516.
- [16] Zhiping Mao, Xuhui Meng, Physics-informed neural networks with residual/gradient-based adaptive sampling methods for solving PDEs with sharp solutions, 2023.
- [17] Yao Li, Shengzhu Shi, Zhichang Guo, Boying Wu, Adversarial training for physics-informed neural networks, 2023.
- [18] Simin Shekarpaz, Mohammad Azizmalayeri, Mohammad Hossein Rohban, PIAT: Physics informed adversarial training for solving partial differential equations, 2022.
- [19] Zheyuan Hu, Khemraj Shukla, George Em Karniadakis, Kenji Kawaguchi, Tackling the curse of dimensionality with physics-informed neural networks, *Neural Netw.* 176 (2024) 106369.
- [20] Jiequn Han, Arnulf Jentzen, E. Weinan, Solving high-dimensional partial differential equations using deep learning, *Proc. Natl. Acad. Sci.* 115 (34) (2018) 8505–8510.
- [21] Maziar Raissi, Forward-backward stochastic neural networks: Deep learning of high-dimensional partial differential equations, 2018.
- [22] Chuwei Wang, Shanda Li, Di He, Liwei Wang, Is L^2 physics-informed loss always suitable for training physics-informed neural network?, 2022.
- [23] Jerome Darbon, Gabriel P. Langlois, Tingwei Meng, Overcoming the curse of dimensionality for some Hamilton–Jacobi partial differential equations via neural network architectures, 2020.
- [24] Jérôme Darbon, Stanley Osher, Algorithms for overcoming the curse of dimensionality for certain Hamilton–Jacobi equations arising in control theory and elsewhere, 2016.
- [25] Zheyuan Hu, Zekun Shi, George Em Karniadakis, Kenji Kawaguchi, Hutchinson trace estimation for high-dimensional and high-order physics-informed neural networks, *Comput. Methods Appl. Mech. Engrg.* 424 (2024) 116883.
- [26] M.F. Hutchinson, A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines, *Comm. Statist. Simulation Comput.* 18 (3) (1989) 1059–1076.

- [27] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, Adrian Vladu, Towards deep learning models resistant to adversarial attacks, 2019.
- [28] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, Anima Anandkumar, Neural operator: learning maps between function spaces with applications to PDEs, *J. Mach. Learn. Res.* 24 (1) (2024).
- [29] Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, Anima Anandkumar, Physics-informed neural operator for learning partial differential equations, 2023.
- [30] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, George Em Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, *Nat. Mach. Intell.* 3 (3) (2021) 218–229.
- [31] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, Anima Anandkumar, Fourier neural operator for parametric partial differential equations, 2021.
- [32] Sam Greydanus, Misko Dzamba, Jason Yosinski, Hamiltonian neural networks, 2019.
- [33] Miles Cranmer, Sam Greydanus, Stephan Hoyer, Peter Battaglia, David Spergel, Shirley Ho, Lagrangian neural networks, 2020.
- [34] Sifan Wang, Shyam Sankaran, Paris Perdikaris, Respecting causality is all you need for training physics-informed neural networks, 2022.
- [35] Levi D. McClenny, Ulisses M. Braga-Neto, Self-adaptive physics-informed neural networks, *J. Comput. Phys.* 474 (2023) 111722.
- [36] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Jonathan Uesato, Pascal Frossard, Robustness via curvature regularization, and vice versa, 2018.
- [37] Maciej Skorski, A modern analysis of hutchinson’s trace estimator, 2020.
- [38] Jorge Nocedal, Stephen J. Wright, Numerical Optimization, second ed., in: Springer Series in Operations Research and Financial Engineering, Springer, 2006.
- [39] Mehdi Dehghan, Finite difference procedures for solving a problem arising in modeling and design of certain optoelectronic devices, *Mathematics and Computers in Simulation* 71 (2006) 1.